

Proactive Multi-Agent Orchestration for Intelligent Engagement on Education



PROJECT REPORT

Submitted by

Group Number: 22

Prakrithi Jain (1BM23CS239)

Sai Pranav Enjeti (1BM23CS286)

Shashank Shantharam Nayak (1BM23CS313)

Siddharth Arya (1BM23CS328)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of

Shreevatsa D S

Assistant Professor, Department of CSE, BMSCE



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU - 560019

Academic Year - 2025–2026

Certificate

This is to certify that the project report entitled **“Proactive Multi-Agent Orchestration for Intelligent Engagement on Education”** is a bonafide work carried out by **Prakrithi Jain (1BM23CS239), Sai Pranav Enjeti (1BM23CS286), Shashank Shantharam Nayak (1BM23CS313), Siddharth Arya (1BM23CS328)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering**, under my supervision during the academic year 2025–2026.

Guide Signature
(Shreevatsa D S)

HoD signature
Dr. Kavitha Sooda

Principal Signature
Dr. Bheemsha Arya

Name and signature of Examiners along with date

Examiner 1

Examiner 2

DECLARATION

We, hereby declare that the Major Project Phase-1 work entitled “Proactive Multi-Agent Orchestration for Intelligent Engagement on Education” is a bonafide work and has been carried out by us under the guidance of Shreevatsa D S, Assistant Professor, Department of Computer Science and Engineering, B.M.S. College of Engineering, Bengaluru, in partial fulfillment of the requirements of the degree of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi. We further declare that, to the best of our knowledge and belief, this project has not been submitted either in part or in full to any other university for the award of any degree.

Candidate details:

SL. NO.	Student Name	USN	Student's Signature
1	Prakrithi Jain	1BM23CS239	
2	Sai Pranav Enjeti	1BM23CS286	
3	Shashank Shantharam Nayak	1BM23CS313	
4	Siddharth Arya	1BM23CS328	

Place: Bengaluru

Date:

Certified that these candidates are students of Computer Science and Engineering Department of B.M.S. College of Engineering. They have carried out the project work of titled “Proactive Multi-Agent Orchestration for Intelligent Engagement on Education” as Major Project Phase-1 work. It is in partial fulfillment for completing the requirement for the award of B.E. degree by VTU. The work is original and I duly certify the same.

Guide Name:

Date:

Signature:

Acknowledgment

We extend our heartfelt appreciation to the Management of BMS College of Engineering and our Principal for offering us the necessary support and facilities for our educational pursuits.

We thank Dr. Kavitha Sooda, Head of the Department, Computer Science and Engineering, for her continual support and for allowing us to conduct this project through the usage of the unique computing facilities provided by the department.

We extend our sincere gratitude to Dr. Nandhini Vineeth and other distinguished members of the review panel. The valuable technical suggestions and reviews made by them helped us in developing the project from a mere idea to a full-fledged project.

We are also grateful to our guide Shreevatsa D S for his valuable contributions during the development of this project. Finally, we thank all our family members, friends, and people who have directly or indirectly helped us in completing this project.

Contents

1	Introduction	1
1.1	Overview	1
1.2	Motivation	1
1.3	Objectives	1
1.4	Scope	2
1.5	SDG Justification	2
1.6	Existing System	2
1.7	Proposed System	2
1.8	Work Plan	3
2	Literature Survey	4
2.1	Overview	4
2.2	Detailed Survey	5
2.2.1	Identification of Research Gaps	6
2.3	Mapping of Objective with Research Gap Identification	6
3	Software and Hardware Requirement Specification	8
3.1	Functional Requirements	8
3.2	Non-functional Requirements	8
3.3	Hardware Requirements	9
3.3.1	For Training and Development	9
3.3.2	Production and Deployment	9
3.4	Software Requirements	9
3.4.1	Development and Machine Learning	10
3.4.2	System Design and Orchestration	10
3.5	Cost Estimation	10
4	Design	11
4.1	High Level Design	11
4.1.1	System Components	12
4.2	Data Layer	12
4.3	System Architecture	12

4.3.1	Ingress and Data Handling	12
4.3.2	Stateful Agentic Orchestration	13
4.3.3	Micro-Agent Interactions	13
4.3.4	The Tri-Tier Data Persistence Strategy	13
5	Implementation	15
5.1	Technologies Used	15
5.2	Details of Implementation	15
5.2.1	AI Orchestrator	15
5.2.2	User Profiling Module	15
6	Summary	17
	REFERENCES	18

List of Figures

4.1	High Level Design	11
4.2	System Architecture	14

List of Tables

2.1	Research Objectives and Corresponding Literature Gaps Mapped	7
4.1	System Data Persistence Layers	14

Abstract

Due to the rapid evolution of AI and LLMs, there is a rising necessity to create highly adaptive and intelligent systems, which can not only respond to user requests but also engage them in proactive and personalized way. Traditionally, AI systems use single model solutions. Such systems provide static response options, have very limited ability to adapt, do not remember any specifics about the user over time. All these factors limit the effectiveness of traditional AI in situations requiring prolonged interaction, personalization and smart decisions.

In this project, we propose creating a proactive multi-agent orchestration solution based on LLMs to engage users in intelligent context-aware communication. This solution will involve several special-purpose agents, including conversational, analytic, recommendation and evaluation agents. They will be part of a system designed to perform specific tasks and then communicate through an orchestration layer, allowing to implement dynamic task management and coordination. An important component of the system will be the user profiling process, which involves capturing user preferences and behavioral characteristics that can be used for providing personalized response options. Our solution will allow proactively initiating interaction with the user, analyzing generated content, such as documents and text data, among others.

Chapter 1 Introduction

1.1 Overview

Contemporary learning systems are changing from a passive approach to an active one with the assistance of Artificial Intelligence. In the study, we present an Intelligent Tutoring System (ITS) using a Multi-Agent Architecture. The proposed system consists of a multiple agents rather than employing a single chatbot. These agents are designed to simulate a one-on-one tutoring session like a private tutor.

1.2 Motivation

The divide in education technology is no longer just about having access to the internet; it is also about having access to quality, personalized education. E-learning systems have been used traditionally as content management systems but fail to address the Two Sigma Problem [1], whereby students receiving individual tutoring outperform those taught in classrooms. We are motivated to provide personalized education by means of autonomous agents that will be able to remember the learner's past, understand what they upload, and help them when problems arise.

1.3 Objectives

Our main goal is to create a system that adheres to the following objectives:

- **Contextual Intelligence:** Build an LLM Orchestration Layer that manages state across sessions using cache and long-term vector memory.
- **Autonomous Assessment:** Create a pipeline that converts raw media (documents, images, videos) into chunked information and generates evaluative questions based on the document.
- **Automated Feedback:** Create a feedback mechanism that produces reports and learning insights based on existing progress in a topic.

- Behavioral Persistence: Design a profiling engine that stores information about the users and their knowledge gaps. This ensures a reduction in the loss of context for the LLM.

1.4 Scope

The current study is concentrated on developing a backend system based on the above objectives.

The system will consist of an API Gateway that ensures secure communication, a data layer based on Vector Databases, and an orchestration layer, where different LLMs will fulfill various functions.

1.5 SDG Justification

This project supports United Nations Sustainable Development Goal 4 (Quality Education), SDG 9 (Industry, Innovation and Infrastructure) and SDG 10 (Reduced Inequalities). By using automation for working on time-consuming tasks such as progress tracking, the system offers an affordable, impactful solution for learners worldwide. It tries to balance the inequalities of society in obtaining quality education by providing a personalized learning companion that is available around the clock. Traditional tutoring services can be costly, posing challenges for learners in low-income communities. To alleviate this, the proposed system provides personalized tutoring capabilities that can be accessed anywhere and anytime.

1.6 Existing System

Current Learning Management Systems (LMS) are mostly static. They may not understand the state of mind of the learner, because they cannot perform one-on-one tutoring. MOOC platforms can only imitate classroom-based learning in an online setting.

Thus, current systems struggle to maintain a coherent conversation over prolonged study periods.

1.7 Proposed System

We propose that a Multi-Agent Orchestration system be adopted. In the center of this system, an LLM Orchestrator will direct user inputs towards specific agents according to their intents.

- The Content Analysis Agent is concerned with parsing of documents, which are needed for processing of study materials.
- The Profiling Agent updates the persistent User Profile Database, accounting for the progress that the learner makes.
- The Question Generation Agent aligns the content with the level of difficulty of the user.
- The Insight Agent recommends the following actions.
- The Proactive Agent interacts with the user via a scheduling capability.

1.8 Work Plan

The process of implementation is planned to follow the following stages:

1. Stage I: Interagent communication and storage schemas of user data development.
2. Stage II: The creation of the API Gateway and Orchestrator.
3. Stage III: Specialized agents' and Vector Database's development and calibration.
4. Stage IV (Improvement): Implementation of the Proactive Scheduler and testing of the system as a whole.

Chapter 2 Literature Survey

A great deal of change has occurred in the education field in recent years; while the conventional way of learning is replaced by MOOCs in the AI era. While it seems easy to access online courses, the problem of personalizing instruction according to the requirements of individual students, “Two Sigma Problem,” remains [1]. Traditionally, digital technologies include videos that are much like those used in the classroom environment, thus lack personalization required for one-to-one tuition. Meanwhile, the implementation of AI and Large Language Models (LLM) can enable the achievement of goals set by Bloom [1]. Despite the good performance demonstrated by LLM in intelligent conversation simulation, its usage currently lacks flexibility. The following literature review will address existing research gaps related to AI education solutions.

This literature review will explore and analyze the existing research gaps regarding AI-based education solutions.

2.1 Overview

The literature review highlights the current status of artificial intelligence in education, evolving from its passive form into a dynamic system. Systematic studies by Chen et al. [2], Elbasi et al. [3] and Agbewali-Koku et al. [4] offer the basis for a taxonomy on machine learning and deep learning approaches. The former can be categorized into proactive approaches, for example, content creation and the latter into reactive approaches, such as performance evaluation [5].

It is observed that the research is shifting away from simple content delivery towards intelligent interactions. The current state reveals that platforms are not only acting as hosts for media but also as systems that monitor students’ activities to prevent them from losing interest in their learning process. This evolution of pedagogy becomes clear in the area of technology, where traditional mathematical models have developed into simulation-based, and now into AI-enabled learning approaches [6], [7].

In the present literature review, the selected papers have been sieved through the following keywords in combination: artificial intelligence in education, machine learning, deep reinforcement learning, student engagement, personalized learning, digital signal processing, education, pedagogy, teaching methods, active learning (AL),

edge AI, edge training, online distillation, real-time training, educational data mining, engaged learning, intelligent tutoring systems, multi-agent system, smart education, hybrid system, large language models.

2.2 Detailed Survey

The literature survey reveals four primary paths of research in the field:

1. Multi Agent Systems:

Traditional e-learning are static in nature. Recent work has focused on Multi-Agent Systems (MAS) to provide dynamic adaptation. Zakaria et al. [8] explains how a multi-agent module (MAT) designed using the Markovian process as the differentiator between different modules. Vuković et al. [9] demonstrates a passive observer that studies the activities of a user.

2. Predictive Analytics and Affective Computing:

The ability to foresee where a student may struggle is actively researched upon in the field of educational A. Bagunaid et al. [10] utilizes deep reinforcement learning model to create early warning systems for student performance, while Rizwan et al. [11] identifies factors affecting student performance in MOOCs through systematic review. Beyond academic scores, Hasan et al. [12] emphasizes the transition toward affective tutoring systems, which incorporate emotional intelligence and student engagement levels [13] into the learning loop. These systems, while computationally expensive to run, provide a framework upon which to build emotionally aware LLMs. Mahafdah et al. [14] uses the data generated through current LMS systems to produce student insights by utilizing ML models like Random Forest, XGBoost, and RNN among many others. Aderghal et al. [15] demonstrates that deep learning architectures, like CNNs and xLSTM, significantly outperform traditional statistical models in capturing sequential learning patterns.

3. Large Language Models and Cognitive Diagnosis:

Another important contribution of LLMs in learning environments is reasoning ability. Zhang et al. [16] proposes SPP-L², which is an efficient approach for performance prediction using learner-generated queries. The proposed framework uses a mixed approach using graph, LLM, and RL models to ensure that the feedback rewards are maximized. The result is better than competing approaches. Similarly, Chen et al. [17] discuss advanced cognitive diagnosis using LLMs to ensure that there is improved understanding by the learners. The authors build a Q-matrix for cognitive diagnosis using the latent knowledge

generated by LLM (Llama3-70B). Some of the other prominent real-world use cases of LLMs include tutoring applications like PETRA [18], which can be used for recursion in dynamic programming problems.

4. Immersive Learning and the Edu-Metaverse:

The notion of space in digital education is evolving with the use of virtual spaces. Illi and Elhassouny [19] and Teichmann [20] have introduced concepts regarding the Edu-Metaverse, which creates immersive virtual learning from physical activities. Such a model seeks to enhance learning by simulating real experiences with high fidelity. Other articles evaluate the efficacy of AI in research-oriented learning approaches [21] and big data [22].

2.2.1 Identification of Research Gaps

Despite this progress, several notable gaps remain:

- **Concept Drift and Memory Loss:** According to Hajmohammed et al. [23], LLMs usually suffer from concept drift because of evolving language use and education environment. Besides, most currently used learning platforms are susceptible to fractured learning because they cannot create persistent memory about students' achievements throughout multiple learning sessions.
- **Knowledge Extraction Using Multiple Types of Input:** Although AI technology allows for analysis of text input, there is no possibility to automate extraction of various types of data including instructional videos and scientific materials, and further transformation of that data into evaluative concepts [6], [24].
- **Fragmented Feedback System:** Although current feedback systems provide general calculation of grades [5], they cannot generate individualized recommendation and analysis in relation to specific user personas and previous experience [12].
- **Scaling Problem of Agent-Based Learning Model:** Even though reinforcement learning algorithms function effectively in multiple agents, Nguyen et al. [25] shows that designing stable agents in changing education context is a big challenge that requires substantial computation capacity [7].

2.3 Mapping of Objective with Research Gap Identification

The primary objective of this work is to develop an intelligent, adaptive learning system that resolves the disconnect between high-level AI theory and practical implementation.

By addressing the identified research gaps, this work focuses on:

Table 2.1: Research Objectives and Corresponding Literature Gaps Mapped

Objective	Proposed Implementation	Mitigated Research Gap	Key References
1. Contextual Intelligence	LLM Orchestrator with short-term caching and long-term vector memory.	Concept drift and temporal memory degradation.	[8], [23]
2. Autonomous Assessment	Pipeline that takes raw media as input and outputs knowledge and questions.	Content bottlenecks and static assessment techniques.	[16], [24]
3. Automated Feedback	Report generation based on persona and affective feedback.	Failure to connect raw analytics and student support.	[12], [22]
4. Behavioral Persistence	Profiling engine for tracking user preferences and knowledge gaps.	Lack of persistent, long-term behavioral tracking in e-learning frameworks.	[9], [15]

The proposed solution thus aims to directly target the Two Sigma Problem by providing a scalable, persistent, and context-aware one-to-one tutoring experience.

Chapter 3 Software and Hardware Requirement Specification

For the successful implementation of an AI-enabled education system, it is important to develop a solid technical base which balances out the need for high-performance training and low-weight production. This chapter discusses the technical requirements needed for supporting the multi-agent system and the LLM-based pipeline.

3.1 Functional Requirements

In order to meet the goals relating to SDG 4, 9, and 10, the system will adhere to the following key functionalities:

- **Real-time Tutoring:** The system will provide instant responses to students' queries by deploying agents that utilize large language models (LLMs).
- **Personalized Learning Paths:** The multi-agent architecture will evaluate student learning outcomes and modify the complexity of the learning content accordingly.
- **User Profiling:** The system will keep track of the activities of users in every session, ensuring that there is always a current profile.
- **Information Retrieval:** The system will provide a function that allows for searching for and retrieving relevant information using vector-based search queries.

3.2 Non-functional Requirements

- **Scalability:** The software architecture must be able to handle an increase in the number of simultaneous users without sacrificing performance.
- **Latency:** For an efficient learning experience, the agent latency must, wherever possible, remain below two seconds.

- **Availability:** The production environment will be accessible through Internet connectivity and have limited downtime.
- **Cost Efficiency:** The development process will employ open-source architectures and cloud computing systems that can be supported by free-tier accounts.

3.3 Hardware Requirements

Hardware requirements are categorized into two different stages: The demanding training stage, and the well-optimized production stage.

3.3.1 For Training and Development

- **Processor:** Intel Core i5 or i7 series (8th gen or above) or AMD Ryzen 5 or 7 series.
- **RAM:** A minimum requirement of 8 GB RAM is needed; 16 GB RAM is highly recommended to facilitate the processing of complex natural language processing models.
- **Graphics Card:** An NVIDIA graphics card of GTX 1650 or higher version is recommended, which supports CUDA operations to enhance the efficiency of model training, or else Google Colab GPU of T4/L4 version can be used.
- **Disk Storage:** The disk storage should be at least 200 GB.
- **Internet Connection:** A fast and stable internet connection is necessary to download pre-trained weights of model files.

3.3.2 Production and Deployment

- **Internet:** A reliable internet connection is necessary to maintain seamless API requests and users' usage.
- **Disk Storage:** Disk storage needs to be at least 10 GB.

3.4 Software Requirements

The software architecture revolves around the AI software stack in the Python ecosystem and modern microservices software stack for the backend.

3.4.1 Development and Machine Learning

- **Programming Language:** Python 3.12 or above for machine learning projects; Java and Python for backend.
- **IDE:** Visual Studio Code, Jupyter Notebook, Google Colab.
- **Machine Learning / Deep Learning Libraries:** TensorFlow, scikit-learn for modeling.
- **NLP Libraries:** Huggingface, SpaCy, and NLTK for natural language processing.
- **Preprocessing / Data Libraries:** Pandas, NumPy.

3.4.2 System Design and Orchestration

- **Multi-Agent Systems:** Consider using LangChain, CrewAI, or AutoGen.
- **Database:** Adopt either NoSQL (MongoDB) or Relational (PostgreSQL) database systems alongside vector databases like FAISS or Qdrant for Retrieval-Augmented Generation (RAG).
- **Backend:** Develop high-performing API endpoints using FastAPI or Quarkus.
- **Deployment:** Create Docker containers and deploy the working model using Hugging Face Spaces.

3.5 Cost Estimation

The economic approach taken in this project focuses on maximizing efficiency and sustainability by leveraging modern open-source technologies to ensure that initial investments are kept to a minimum. Expected costs include the following:

1. **Compute:** Code development is done using institution-based systems, as well as Google Colab, which has a free tier option.
2. **Hosting:** Hosting is kept light by utilizing the free tier of Hugging Face Spaces.
3. **Software:** All software requirements are covered by open-source software (Python, FastAPI, LangChain).

Estimated Total Cost: Minimal, if any. Costs will be incurred for computing power in case of an increase in users beyond what is available in the free tier.

Chapter 4 Design

4.1 High Level Design

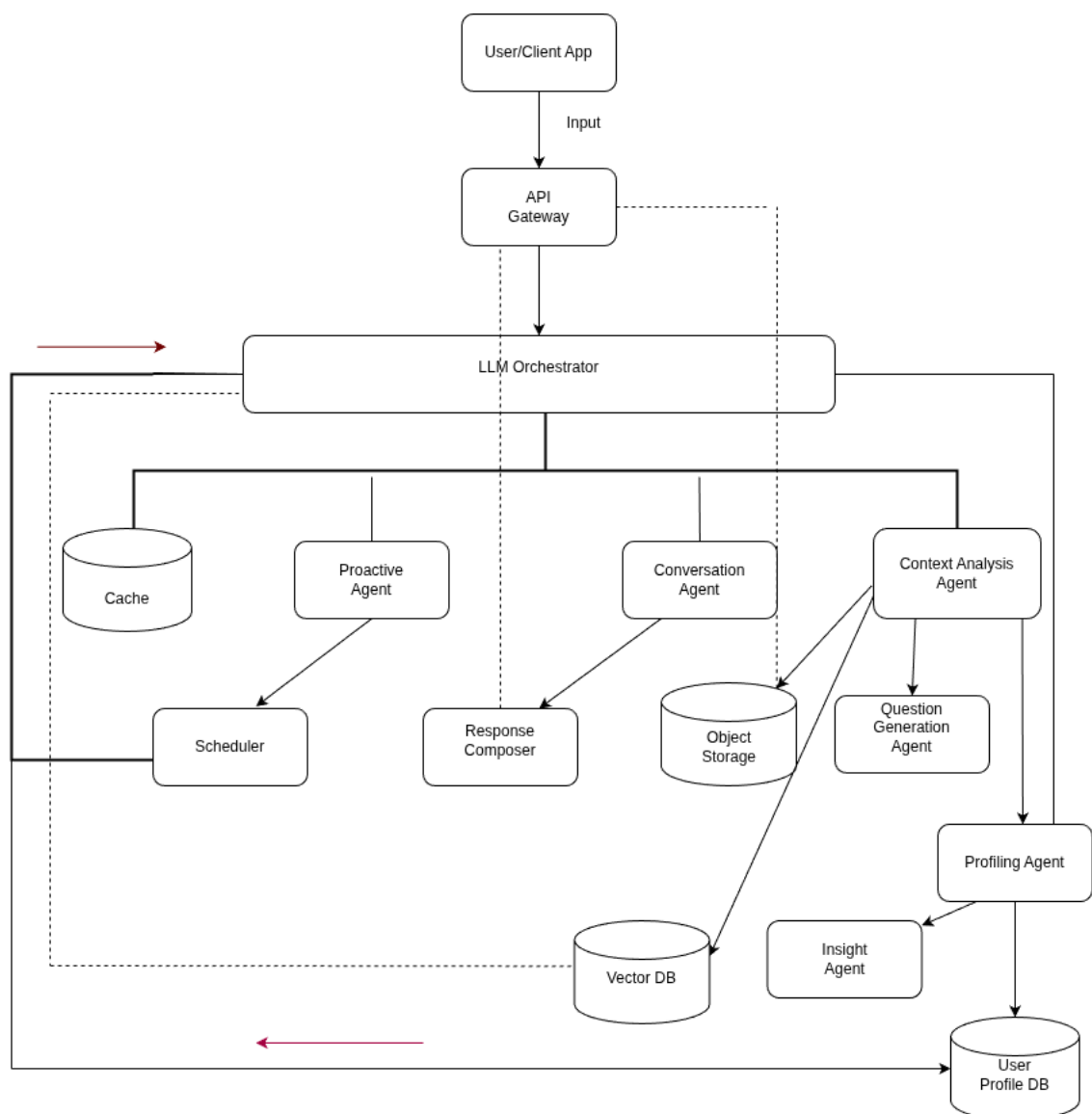


Figure 4.1: High Level Design

4.1.1 System Components

API Gateway

The gateway is using FastAPI to handle JWT authentication and route file streams to AWS S3/MinIO.

LLM Orchestrator

The orchestrating agent uses LangGraph to route requests between agents, accessing Redis to keep track of sessions.

Specialized Agents

- **Profiling Agent:** Ensures the syncing of behavioral data in PostgreSQL.
- **Content Analysis Agent:** Converts raw files into vectors searchable by Qdrant.
- **Question Generation Agent:** Composes assessments based on content vectors.

4.2 Data Layer

The proposed framework incorporates a three tier approach to data storage: PostgreSQL as a relational database for users' information, Qdrant as a semantic vector index, and Redis as session cache.

4.3 System Architecture

The shift in the system architecture from a traditional request-response interaction to a stateful agency-based interaction ensures that the Large Language Model (LLM) provides not just generated output, but also acts as the processing center interacting with microservices and data layers to deliver the customized learning pathway.

4.3.1 Ingress and Data Handling

The entry point of the system is a high-performance **API Gateway** implemented via **FastAPI**. It manages asynchronous request handling and implements OAuth2-based JWT authentication. To ensure system scalability, the gateway utilizes a decoupled file handling strategy:

- **Object Storage Integration:** Large binary objects (PDFs, media files) are offloaded directly to **AWS S3** or **MinIO**.

- **Metadata Routing:** Only the metadata and object references are passed to the downstream agents, reducing the memory footprint of the orchestrator.

4.3.2 Stateful Agentic Orchestration

At the core of the intelligence layer lies the **LLM Orchestrator**, built using **LangGraph**. Unlike standard chain-based architectures, this graph-based approach allows for cyclic transitions and complex decision-making.

The orchestrator maintains a state object, S , defined as:

$$S = \{H_c, U_p, C_{ret}, T_{meta}\}$$

Where H_c represents the conversation history, U_p the user profile attributes, C_{ret} the context retrieved from vector storage, and T_{meta} the task-specific metadata. To maintain low-latency interactions, the **Short-Term Memory (STM)** is persisted in **Redis**, allowing the orchestrator to resume states across different API calls without re-processing the entire history.

4.3.3 Micro-Agent Interactions

The architecture employs a *Specialized Agent Cluster* where each node is a fine-tuned or prompt-engineered LLM instance:

- **Profiling Agent (PA):** Executes latent trait extraction. It monitors interactions for shifts in user proficiency and updates the **PostgreSQL** database via atomic UPSERT operations to ensure data integrity.
- **Content Analysis Agent (CAA):** Triggers a Retrieval-Augmented Generation (RAG) pipeline. It extracts text from S3 objects, performs semantic chunking, and generates high-dimensional embeddings for storage in **Qdrant**.
- **Question Generation Agent (QGA):** Synthesizes assessments by performing a semantic query on the **Vector DB** and cross-referencing the user's current knowledge level from the **User Profile DB**.

4.3.4 The Tri-Tier Data Persistence Strategy

The system architecture utilizes three distinct storage paradigms to optimize for consistency, speed, and semantic relevance:

This modular decoupling ensures that individual agents can be scaled or updated independently without disrupting the global orchestration logic, fulfilling the requirement for a resilient and adaptive learning environment.

Layer	Technology	Technical Utility
Cache / STM	Redis	Session persistence and graph state management.
Relational / LTM	PostgreSQL	ACID-compliant storage for user metrics and profiles.
Vector Knowledge	Qdrant	HNSW-indexed semantic search for educational content.

Table 4.1: System Data Persistence Layers

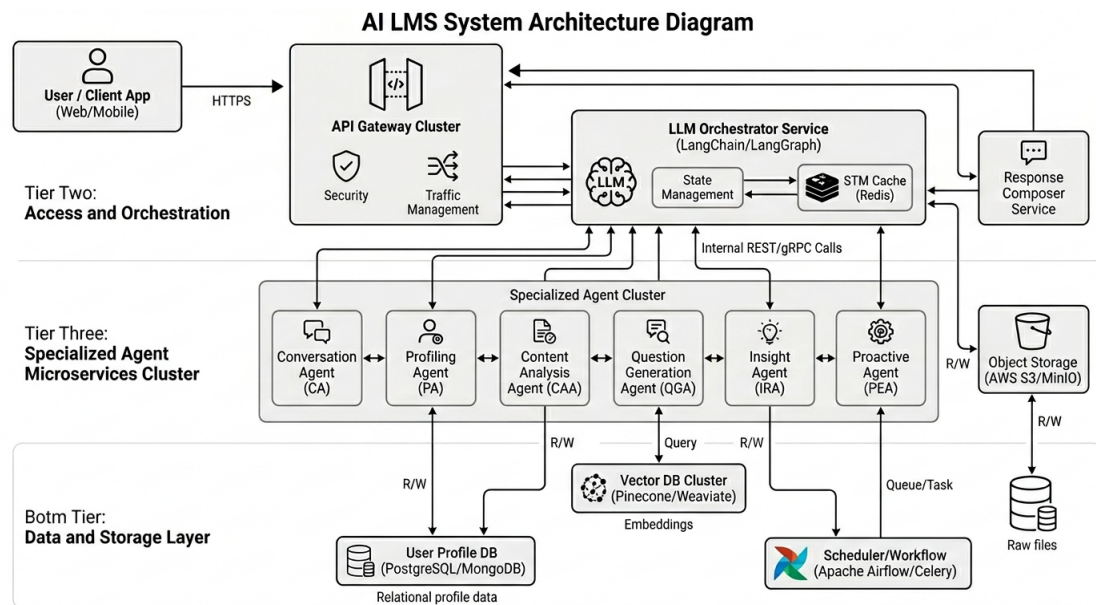


Figure 4.2: System Architecture

Chapter 5 Implementation

5.1 Technologies Used

The suggested implementation will leverage an advanced AI stack consisting of **FastAPI**, a high-performance Python framework for building APIs; **LangGraph**, a language model orchestration tool; **Qdrant**, a high-performance vector database; **PostgreSQL**, a robust relational database for user metadata; and **Redis**, a high-performance key-value store for session caching.

5.2 Details of Implementation

5.2.1 AI Orchestrator

The AI Orchestrator module can be considered a state-driven workflow managed by **LangGraph**. The main purpose of this component is to control the execution flow by utilizing the global `AgentState`. This object contains the message history, user profiles, and other information needed by agents to complete their tasks.

- **State Management:** Implements a `TypedDict` structure that guarantees the availability of required data (e.g., knowledge level, retrieved documents) in every agent.
- **Routing Logic:** Leverages conditional routing techniques, which dynamically adjust the workflow depending on the `current_task` (e.g., switching to `question_gen` for assessments).
- **Integration with External Systems:** Uses **Redis** to store session-related information for low-latency response generation.

5.2.2 User Profiling Module

The above module is implemented via the `profiling_agent`, which offers a standardized model for the learner at all times.

- **Dynamic Profiling:** It relies on function calling within LLMs to interpret interactions and change the `knowledge_level`.
- **Persistence Mechanism:** The `user_profiles` PostgreSQL table is updated with UPSERT statements for unified management.

Chapter 6 Summary

Two Sigma problem constitutes the height of education, however, as we have gone away from conventional classrooms to contemporary LLMs, it turns out to be quite disjointed. There have been efforts made to come up with solutions that although are very effective, suffer from drift in context and are incapable of remembering the particularities of each individual student once the session is over. This survey makes it clear that although Multi-Agent Systems and LLMs have helped to reach places never imagined before, the most crucial task now is the one that involves the connection between the user and AI.

REFERENCES

- [1] B. S. Bloom, “The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring,” *Educational researcher*, vol. 13, no. 6, pp. 4–16, 1984.
- [2] L. Chen, P. Chen, and Z. Lin, “Artificial intelligence in education: A review,” *IEEE Access*, vol. 8, pp. 75 264–75 278, 2020.
- [3] E. Elbasi, M. Nadeem, Y. I. Alzoubi, A. E. Topcu, and G. Varghese, “Machine learning in education: Innovations, impacts, and ethical considerations,” *IEEE Access*, 2025.
- [4] C. E. K. Agbewali-Koku, M. A. Rahman, M. Hamada, M. A. Ali, L. N. Oysharja, and M. T. Hossain, “A systematic review of machine learning techniques in online learning platforms,” in *2022 IEEE 15th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*, IEEE, 2022, pp. 247–250.
- [5] S. Mallik and A. Gangopadhyay, “Proactive and reactive engagement of artificial intelligence methods for education: A review,” *Frontiers in Artificial Intelligence*, vol. 6, p. 1 151 391, 2023.
- [6] M. B. López, “Evolving pedagogy in digital signal processing education: Ai-assisted review and analysis,” *IEEE Access*, vol. 13, pp. 45 559–45 567, 2025.
- [7] M. Boldo, M. De Marchi, E. Martini, S. Aldegheri, and N. Bombieri, “Domain-adaptive online active learning for real-time intelligent video analytics on edge devices,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 11, pp. 4105–4116, 2024.
- [8] R. Zakaria, B. Hadhoum, E. Mourad, and M. Mustapha, “Dynamic adaptation in an intelligent multi-tutoring system: A multi-agent approach,” *IEEE Access*, 2025.
- [9] I. Vuković et al., “Multi-agent system observer: Intelligent support for engaged e-learning,” *Electronics*, vol. 10, no. 12, p. 1370, 2021.

- [10] W. Bagunaïd, N. Chilamkurti, A. S. Shahraki, and S. Bamashmos, "Visual data and pattern analysis for smart education: A robust drl-based early warning system for student performance prediction," *Future Internet*, vol. 16, no. 6, p. 206, 2024.
- [11] S. Rizwan, C. K. Nee, and S. Garfan, "Identifying the factors affecting student academic performance and engagement prediction in mooc using deep learning: A systematic literature review," *IEEE Access*, vol. 13, pp. 18 952–18 982, 2025.
- [12] M. A. Hasan, N. F. M. Noor, S. S. A. Rahman, and M. M. Rahman, "The transition from intelligent to affective tutoring system: A review and open issues.," *IEEE Access*, vol. 8, no. 8, pp. 204 612–204 638, 2020.
- [13] S. Ayouni, F. Hajjej, M. Maddeh, and S. Al-Otaibi, "A new ml-based approach to enhance student engagement in online environment," *Plos one*, vol. 16, no. 11, e0258788, 2021.
- [14] R. Mahafdah, S. Bouallegue, and R. Bouallegue, "Enhancing e-learning through ai: Advanced techniques for optimizing student performance," *PeerJ Computer Science*, vol. 10, e2576, 2024.
- [15] Y. Aderghal, H. Hafidi, and A. El Ghazi, "Xlstmkt: Xlstm for knowledge tracing," *IEEE Access*, 2025.
- [16] Z. Zhang, X. He, and Y. Zhang, "Spp-l²: A ppo-enhanced large language model framework for student performance prediction on learnersourced questions," *IEEE Access*, 2025.
- [17] X. Chen, J. Zhang, T. Zhou, and F. Zhang, "Llm-cdm: A large language model enhanced cognitive diagnosis for intelligent education," *IEEE Access*, 2025.
- [18] D. W. Hewedy and A. S. Abdelhady, "Petra: A personalized educational tutoring tool for recursive assistance leveraging multi-model llms for dynamic programming learning," in *2025 15th International Conference on Electrical Engineering (ICEENG)*, IEEE, 2025, pp. 1–6.
- [19] C. Illi and A. Elhassouny, "Edu-metaverse: A comprehensive review of virtual learning environments," *IEEE Access*, 2025.
- [20] M. R. Teichmann, "How to design immersive virtual learning environments based on real-world processes for the edu-metaverse—a design process framework," *IEEE Transactions on Learning Technologies*, vol. 18, pp. 100–118, 2025.
- [21] F. Festiyed, D. Desnita, Z. Natasya, M. A. Fadillah, and F. Novitra, "From assistance to autonomy: Ai integration in structured research-based learning for higher education," *Electronic Journal of e-Learning*, vol. 24, no. 1, pp. 109–124, 2026.

- [22] A. Bakhouyi, A. Dehbi, L. Amhaimar, S. Broumi, M. Talea, and A. Khalidi, “Improving smart learning: Course completion via ai-driven hybrid system integration in big data,” *Telematics and Informatics Reports*, vol. 18, p. 100 199, 2025.
- [23] M. Hajmohammed, P. Chountas, and T. J. Chaussalet, “Concept drift in large language models: Challenges of evolving language, contexts, and the web,” in *2025 1st International Conference on Computational Intelligence Approaches and Applications (ICCIAA)*, IEEE, 2025, pp. 1–6.
- [24] N. Pradeesh, M. Thushara, K. A. Krishna, V. Pranav, and S. Krishnamoorthy, “Ai-driven personalized learning and remedial recommendation through knowledge concept-centric evaluation,” *IEEE Access*, 2025.
- [25] T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, “Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications,” *IEEE Transactions on Cybernetics*, vol. 50, no. 9, pp. 3826–3839, 2020.